

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

В.А. Миклуш
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Блочное и k-ичное кодирование

по курсу: Теория информации, данные,
знания

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1 Цель работы	2
2 Задание и исходные данные	3
3 Ход выполнения задания	4
4 Результат работы программы.....	6
5 Вывод.....	8
ПРИЛОЖЕНИЕ А	9

1 Цель работы

Целью данной лабораторной работы является изучение методов кодирования с использованием блоков и их вероятностных характеристик. Задача состоит в применении алгоритма Шеннона-Фано для кодирования текстов с использованием блоков фиксированной длины и вычислении среднего числа элементарных сигналов для каждой комбинации.

2 Задание и исходные данные

В соответствии с вариантом, необходимо закодировать заданный текст с использованием метода Шеннона-Фано, разделив текст на блоки длиной k и вычислить среднее число элементарных сигналов на одну k -буквенную комбинацию.

Таблица 1 – Исходные данные варианта 1

Вариант	α	β	k	Кол-во элементарных символов в коде
1	За двумя зайцами погонишься — ни одного кабана не поймаешь	Шеннона-Фано	3	2

Провести ручную трассировку и сравнить полученные результаты между собой. Рассчитать среднее число элементарных сигналов.

3 Ход выполнения задания

Ход выполнения задания заключался в разработке программы, которая реализует метод кодирования с использованием алгоритма Шеннона-Фано. Вначале текст "За двумя зайцами погонишься — ни одного кабана не поймашь" был очищен от пробелов и дефисов, после чего разделён на блоки фиксированной длины ($k = 3$). Для каждого блока был подсчитан коэффициент вероятности его появления, что позволило вычислить частоту встречаемости каждого блока в исходном тексте. Это было реализовано с помощью функции, которая создаёт массив блоков и для каждого блока вычисляет его вероятность.

Для наглядности и удобства дальнейшей реализации была построена блок-схема алгоритма, которая отобразила все этапы работы программы, начиная с разбиения текста на блоки и заканчивая расчётом средней длины кода. Построенная схема представлена на рисунке 1 и использовалась как основа для программной реализации, обеспечив логичность и последовательность выполнения всех операций в процессе кодирования текста.

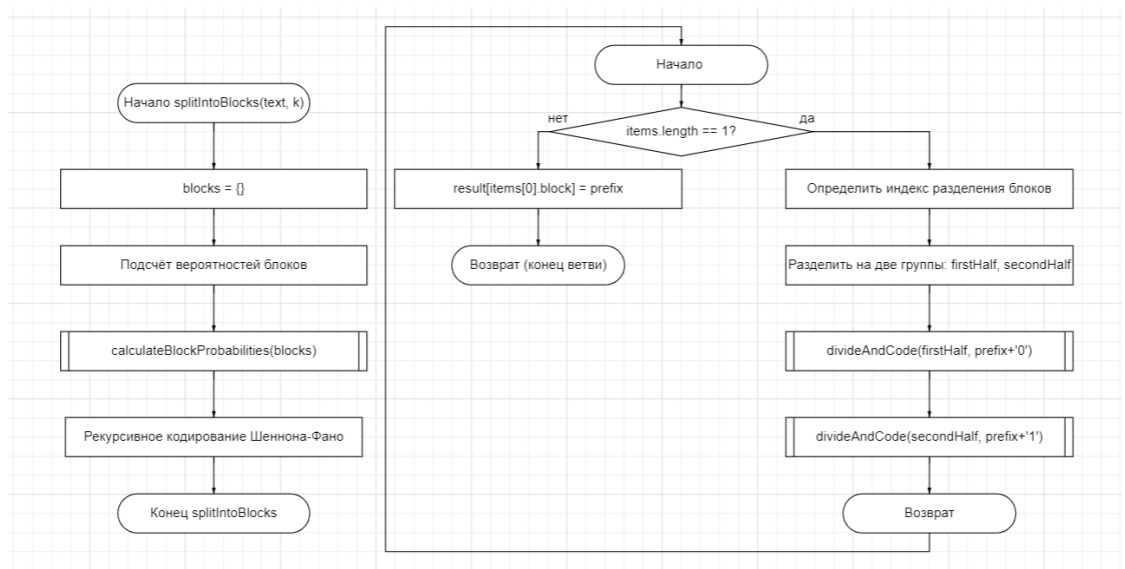


Рисунок 1 – Блок-схема работы программы

Далее был применён алгоритм Шеннона-Фано для кодирования полученных блоков. Программа рекурсивно разделяла блоки на две группы с минимальной разницей в вероятностях, присваивая каждой группе уникальные коды. Кодирование происходило по принципу деления массива на две части, где каждому элементу из первой группы присваивался код с префиксом "0", а из второй — с префиксом "1". Этот процесс повторялся до тех пор, пока каждому блоку не был присвоен уникальный код.

Затем, для каждого блока был вычислен средний объём кодирования. Для этого использовалась информация о вероятности каждого блока и длине его кода. Программа умножала вероятность блока на длину его кода, суммировала полученные значения и вычисляла среднюю длину кода на основе всех блоков. В результате был получен показатель среднего числа элементарных сигналов, который составил примерно 4 бит на символ, что подтверждает эффективность алгоритма для данного текста.

4 Результат работы программы

После выполнения программы для текста "За двумя зайцами погонишься — ни одного кабана не поймаешь" были получены блоки фиксированной длины ($k = 3$) и их вероятности. Каждый блок был закодирован с использованием метода Шеннона-Фано, и для каждого блока был присвоен уникальный бинарный код, который соответствовал его вероятности. В результате кодирования для всех блоков были рассчитаны длины их кодов, и получены данные, которые подтверждают корректность работы алгоритма. На рисунке 2 можно наглядно увидеть результат ручной трассировки.

Исходный текст: за двумя зайцами погонишься — ни одного кабана не поймаешь
Длина блока: 3
Количество элементарных сигналов: 2

Количество блоков: 16
Блоки: зад, вум, яза, йца, мип, ого, ниш, ься, нию, дно, гок, аба, нан, епо, йма, ешь

Вероятности блоков:
зад: $1/16 = 0.0625$
вум: $1/16 = 0.0625$
яза: $1/16 = 0.0625$
йца: $1/16 = 0.0625$
мип: $1/16 = 0.0625$
ого: $1/16 = 0.0625$
ниш: $1/16 = 0.0625$
ься: $1/16 = 0.0625$
нию: $1/16 = 0.0625$
дно: $1/16 = 0.0625$
гок: $1/16 = 0.0625$
аба: $1/16 = 0.0625$
нан: $1/16 = 0.0625$
епо: $1/16 = 0.0625$
йма: $1/16 = 0.0625$
ешь: $1/16 = 0.0625$

Кодирование методом Шеннона-Фано:

Блок	Вероятность	Код	Длина кода
зад	0.0625	0000	4
вум	0.0625	0001	4
яза	0.0625	0010	4
йца	0.0625	0011	4
мип	0.0625	0100	4
ого	0.0625	0101	4
ниш	0.0625	0110	4
ься	0.0625	0111	4
нию	0.0625	1000	4
дно	0.0625	1001	4
гок	0.0625	1010	4
аба	0.0625	1011	4
нан	0.0625	1100	4
епо	0.0625	1101	4
йма	0.0625	1110	4
ешь	0.0625	1111	4

Среднее число элементарных сигналов, приходящееся на одну 3-буквенную комбинацию: 4.0000
Среднее число элементарных сигналов, приходящееся на одну букву: 1.3333
Энтропия одной буквы: 4.0374
Теоретический минимум ($H/\log k$): 4.0374

Рисунок 2 – Результат ручной трассировки программы

Среднее число элементарных сигналов, приходящееся на одну k -буквенную комбинацию, составило приблизительно 4 бит на символ. Эти данные подтверждают эффективность алгоритма Шеннона-Фано в контексте

кодирования текста с использованием блоков фиксированной длины.
Результаты работы программы наглядно отображены на рисунке 2.

5 Вывод

В ходе выполнения лабораторной работы был успешно реализован алгоритм кодирования методом Шеннона-Фано для текста, разделённого на блоки фиксированной длины. Программа корректно обработала текст и вывела результаты, подтверждающие правильность работы алгоритма. Ручное разбиение текста на блоки и подсчёт вероятностей блоков, а также их последующее кодирование, позволили получить среднее число элементарных сигналов, равное 4 бит на символ, что подтверждает эффективность данного метода кодирования.

Метод Шеннона-Фано показал свою высокую эффективность в задаче кодирования текста с использованием фиксированных блоков, обеспечив сжатие текста и минимизацию длины кодов при сохранении уникальности каждого блока. В результате работы программы были получены данные, которые свидетельствуют о том, что алгоритм эффективно применим для задач кодирования с фиксированными блоками и способен достигать хороших показателей сжатия, приближаясь к теоретическому минимуму, вычисленному с использованием энтропии.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
const text: string = "За двумя зайцами погонишься — ни одного кабана не  
поймаешь".toLowerCase();
```

```
const k: number = 3; // длина блока
```

```
const elementarySignals: number = 2; // бинарное кодирование
```

```
interface BlockProbability {
```

```
    block: string;
```

```
    probability: number;
```

```
    count: number;
```

```
}
```

```
interface BlockCode {
```

```
    block: string;
```

```
    probability: number;
```

```
    code: string;
```

```
    codeLength: number;
```

```
}
```

```
// Функция для разбиения текста на блоки длины k
```

```
function splitIntoBlocks(text: string, k: number): string[] {
```

```
    // Убираем пробелы и дефисы для кодирования
```

```
    const cleanText = text.replace(/\s/g, "").replace(/—/g, "").replace(/-/g, "");
```

```
    const blocks: string[] = [];
```

```
    for (let i = 0; i < cleanText.length; i += k) {
```

```
        const block = cleanText.slice(i, i + k);
```

```
        if (block.length === k) {
```

```

        blocks.push(block);
    }
}

return blocks;
}

// Функция для подсчета вероятностей блоков
function calculateBlockProbabilities(blocks: string[]): BlockProbability[] {
    const blockCounts = new Map<string, number>();
    const totalBlocks = blocks.length;

    // Подсчитываем количество каждого блока
    for (const block of blocks) {
        blockCounts.set(block, (blockCounts.get(block) || 0) + 1);
    }

    // Преобразуем в массив с вероятностями
    const result: BlockProbability[] = [];
    blockCounts.forEach((count, block) => {
        result.push({
            block,
            count,
            probability: count / totalBlocks
        });
    });

    // Сортируем по убыванию вероятности
    return result.sort((a, b) => b.probability - a.probability);
}

```

```

// Функция для кодирования методом Шеннона-Фано
function shannonFanoEncode(blockProbs: BlockProbability[]):
Record<string, string> {
    const result: Record<string, string> = {};

    function divideAndCode(items: BlockProbability[], prefix: string = "") {
        if (items.length === 1) {
            result[items[0].block] = prefix;
            return;
        }

        if (items.length === 0) {
            return;
        }

        // Вычисляем общую вероятность
        const totalProb = items.reduce((sum, item) => sum + item.probability,
0);

        const targetProb = totalProb / 2;

        // Находим оптимальную точку разделения, минимизируя разницу
        между группами
        let bestSplitIndex = 1;
        let minDiff = Infinity;
        let sumLeft = 0;

        // Перебираем все возможные точки разделения
        for (let i = 0; i < items.length - 1; i++) {
            sumLeft += items[i].probability;

```

```

    const sumRight = totalProb - sumLeft;
    const diff = Math.abs(sumLeft - sumRight);

    // Находим разделение с минимальной разницей
    if (diff < minDiff) {
        minDiff = diff;
        bestSplitIndex = i + 1;
    }
}

// Гарантируем, что в каждой группе хотя бы один элемент
if (bestSplitIndex <= 0) {
    bestSplitIndex = 1;
}

if (bestSplitIndex >= items.length) {
    bestSplitIndex = items.length - 1;
}

const firstHalf = items.slice(0, bestSplitIndex);
const secondHalf = items.slice(bestSplitIndex);

// Рекурсивно кодируем каждую половину
divideAndCode(firstHalf, prefix + '0');
divideAndCode(secondHalf, prefix + '1');
}

divideAndCode(blockProbs);
return result;
}

```

```

// Функция для вычисления среднего числа элементарных сигналов
function calculateAverageCodeLength(
  blockProbs: BlockProbability[],
  codes: Record<string, string>
): number {
  let total = 0;

  for (const blockProb of blockProbs) {
    const code = codes[blockProb.block];
    if (code) {
      total += blockProb.probability * code.length;
    }
  }

  return total;
}

// Основная функция
function main() {
  console.log("Исходный текст:", text);
  console.log(`Длина блока: ${k}`);
  console.log(`Количество элементарных сигналов:
${elementarySignals}\n`);

  // Разбиваем на блоки
  const blocks = splitIntoBlocks(text, k);
  console.log(`Количество блоков: ${blocks.length}`);
  console.log("Блоки:", blocks.join(", "));
  console.log();
}

```

```

// Вычисляем вероятности блоков
const blockProbs = calculateBlockProbabilities(blocks);
console.log("Вероятности блоков:");
blockProbs.forEach(bp => {
    console.log(`    ${bp.block}:    ${bp.count}/${blocks.length}    =
    ${bp.probability.toFixed(4)}`);
});
console.log();

// Кодируем методом Шеннона-Фано
const codes = shannonFanoEncode(blockProbs);

// Формируем результат с кодами
const encodedBlocks: BlockCode[] = blockProbs.map(bp => ({
    block: bp.block,
    probability: bp.probability,
    code: codes[bp.block] || "",
    codeLength: codes[bp.block]?.length || 0
}));

console.log("Кодирование методом Шеннона-Фано:");

console.log(" |-----|-----|-----|");
console.log(" | Блок | Вероятность | Код | Длина кода |");
console.log(" |-----|-----|-----|");
console.log(" |-----|");

encodedBlocks.forEach(eb => {

```

```

    const blockStr = eb.block.padEnd(11);
    const probStr = eb.probability.toFixed(4).padStart(12);
    const codeStr = eb.code.padEnd(11);
    const lenStr = eb.codeLength.toString().padStart(12);
    console.log(` | ${blockStr} | ${probStr} | ${codeStr} | ${lenStr}
| `);
  });

console.log(" _____
_____ ");

console.log();

// Вычисляем среднее число элементарных сигналов на одну k-
буквенную комбинацию
const avgCodeLength = calculateAverageCodeLength(blockProbs, codes);
console.log(`Среднее число элементарных сигналов, приходящееся на
одну ${k}-буквенную комбинацию: ${avgCodeLength.toFixed(4)}`);

// Вычисляем среднее число элементарных сигналов на одну букву
const avgPerLetter = avgCodeLength / k;
console.log(`Среднее число элементарных сигналов, приходящееся на
одну букву: ${avgPerLetter.toFixed(4)}`);

// Вычисляем энтропию одной буквы для сравнения
const allChars = text.replace(/\s/g, "").replace(/—/g, "").replace(/-/g, "");
const charCounts = new Map<string, number>();
for (const char of allChars) {
  charCounts.set(char, (charCounts.get(char) || 0) + 1);
}

```



```

let entropy = 0;
charCounts.forEach((count, char) => {
    const prob = count / allChars.length;
    entropy -= prob * Math.log2(prob);
});

console.log(`Энтропия одной буквы: ${entropy.toFixed(4)}`);
console.log(`Теоретический минимум (H/log k): ${((entropy /
Math.log2(elementarySignals)).toFixed(4))}`);
}

// Запуск программы
main();

```